

Coming Up with Recursive Algorithms

Hopefully, this chapter has given you a firm idea of how recursive functions compare to the iterative algorithms you're likely more familiar with. The rest of this book dives into the details of various recursive algorithms. But how should you go about writing your own recursive functions?

The first step is always to identify the recursive case and the base case. You can take a top-down approach by breaking the problem into subproblems that are similar to the original problem but smaller; this is your *recursive case*. Then consider when the subproblems are small enough to have a trivial answer; this is your *base case*. Your recursive function may have more than one recursive case or base case, but all recursive functions will always have at least one recursive case and at least one base case.

The recursive Fibonacci algorithm is an example. A Fibonacci number is the sum of the previous two Fibonacci numbers. We can break the problem of finding a Fibonacci number into the subproblems of finding two smaller Fibonacci numbers. We know the first two Fibonacci numbers are both 1, so that provides the base case answer once the subproblems are small enough.

Sometimes it helps to take a bottom-up approach and consider the base case first, and then see how larger and larger problems are constructed and solved from there. The recursive factorial problem is an example. The factorial of $1!$ is 1. This forms the base case. The next factorial is $2!$, and you create it by multiplying $1!$ by 2. The factorial after that, $3!$, is created by multiplying $2!$ by 3, and so on. From this general pattern, we can figure out what the recursive case for our algorithm will be.